

Chapter 3 Lab – Encoder-Decoder Translation

Advanced Machine Learning for Finance

Giovanni Della Lunga

Master's Degree in Quantitative Finance – University of Bologna

Bologna – 2026

Section 1

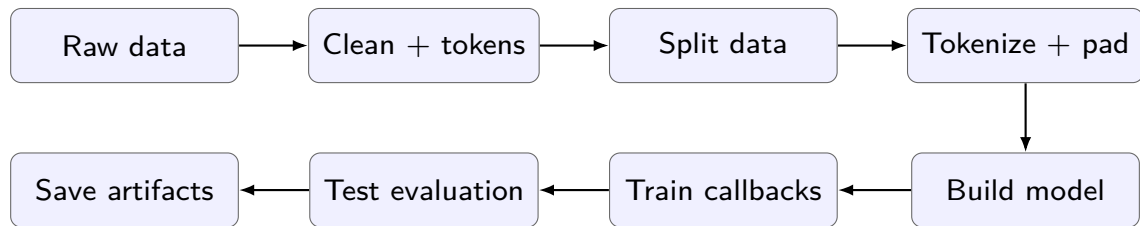
Lost in Translation

Notebooks: `06_encoder-decoder_translation_with_keras.ipynb`

The notebook implements a small neural machine translation system.

Its goal is **educational**: it shows the basic mechanics of an LSTM-based encoder-decoder model before introducing more advanced architectures such as attention, and Transformers.

Notebook workflow



What problem are we solving?

The model learns a mapping from a Spanish sentence to an English sentence.

Let

$$x = (x_1, x_2, \dots, x_{T_s})$$

be the Spanish input sequence, where:

- x_t is the token identifier at Spanish position t ;
- T_s is the maximum Spanish sequence length after padding.

Let

$$y = (y_1, y_2, \dots, y_{T_e})$$

be the English target sequence, where:

- y_t is the target English token identifier at output position t ;
- T_e is the maximum English sequence length after padding.

Imports and reproducibility

The notebook imports TensorFlow/Keras for neural networks, NumPy for arrays, and Pickle/JSON for saving artifacts.

```
1 import os
2 import json
3 import pickle
4 import random
5 from pathlib import Path
6
7 import numpy as np
8 import tensorflow as tf
9
10 from tensorflow.keras.preprocessing.text import Tokenizer
11 from tensorflow.keras.utils import pad_sequences
12 from tensorflow.keras.models import Model, load_model
13 from tensorflow.keras.layers import Input, Embedding, Dense, LSTM
14 from tensorflow.keras.layers import RepeatVector, TimeDistributed, Activation
```

Reproducibility settings

The notebook fixes random seeds:

```
1 RANDOM_SEED = 42
2 random.seed(RANDOM_SEED)
3 np.random.seed(RANDOM_SEED)
4 tf.random.set_seed(RANDOM_SEED)
```

This makes the experiment more reproducible, although perfect reproducibility may still depend on:

- hardware;
- GPU kernels;
- TensorFlow version;
- low-level numerical libraries.

Configuration block

The notebook exposes the main choices in one place.

```
1 START_ROW = 1000
2 END_ROW = 80000
3
4 VAL_SIZE = 0.10
5 TEST_SIZE = 0.10
6
7 BATCH_SIZE = 64
8 EPOCHS = 50
9 LEARNING_RATE = 1e-3
10
11 EMBEDDING_DIM = 128
12 LATENT_DIM = 64
13 DROPOUT_RATE = 0.20
```

This makes the experiment easier to modify.

Data format

The notebook expects a tab-separated bilingual file.

A typical row has the form:

```
English sentence \t Spanish sentence
```

The code uses only the first two columns even if additional attribution columns are present.

This is a common structure for small translation datasets used in introductory sequence-to-sequence examples.

Loading and parsing the file

The notebook checks whether the file exists, reads all lines, and extracts valid English-Spanish pairs.

```
1 with DATA_PATH.open("r", encoding="utf-8") as translation_file:
2     raw_data = translation_file.read().splitlines()
3
4 raw_pairs = []
5 for row in raw_data:
6     parts = row.split("\t")
7     if len(parts) >= 2:
8         english = parts[0].strip()
9         spanish = parts[1].strip()
10        if english and spanish:
11            raw_pairs.append((english, spanish))
```

This produces a list of pairs such as:

(English sentence, Spanish sentence).

Text cleaning

The cleaning function lowercases each sentence and removes punctuation.

```
1 def clean_sentence(sentence: str) -> str:
2     sentence = sentence.lower().strip()
3     punctuation_to_remove = string.punctuation + "SPANISH_INVERTED_MARKS"
4     sentence = sentence.translate(
5         str.maketrans("", "", punctuation_to_remove)
6     )
7     sentence = " ".join(sentence.split())
8     return sentence
```

This is deliberately simple. For a real translation system, punctuation should usually be handled more carefully because it carries syntactic information.

Start and end tokens

The notebook adds explicit boundary tokens to each English target sentence.

```
1 START_TOKEN = "<start>"
2 END_TOKEN = "<end>"
3
4 english_target = f"{START_TOKEN} {english_clean} {END_TOKEN}"
```

These tokens teach the model that a target sentence has boundaries:

- `<start>` marks the beginning of the target sequence;
- `<end>` marks the end of the target sequence.

They are especially important in more advanced decoders that generate one token at a time.

Why train-validation-test split?

The notebook uses:

- **training set**: used to learn model weights;
- **validation set**: used during training to monitor generalization and stop early;
- **test set**: used only once for final evaluation.

The test set simulates genuinely unseen examples.

Splitting indices

The split is created by randomly permuting the example indices.

```
1 def split_indices(n_samples, val_size, test_size, seed=42):
2     rng = np.random.default_rng(seed)
3     indices = rng.permutation(n_samples)
4
5     n_test = int(round(n_samples * test_size))
6     n_val = int(round(n_samples * val_size))
7
8     test_idx = indices[:n_test]
9     val_idx = indices[n_test:n_test + n_val]
10    train_idx = indices[n_test + n_val:]
11
12    return train_idx, val_idx, test_idx
```

The random seed makes the split reproducible.

Tokenizer fitting without leakage

The notebook fits tokenizers only on the training set.

This matters because validation and test data should remain unseen during model preparation.

If a tokenizer is fitted on the full dataset, then information from the validation and test sets leaks into preprocessing.

Unknown words in validation or test data are mapped to:

`<unk>`

which stands for *unknown token*.

Creating Keras tokenizers

The notebook creates one tokenizer for Spanish and one for English.

```
1 def create_tokenizer(texts, oov_token=UNK_TOKEN):
2     tokenizer = Tokenizer(oov_token=oov_token)
3     tokenizer.fit_on_texts(texts)
4     return tokenizer
5
6 spa_text_tokenizer = create_tokenizer(spa_train_text)
7 eng_text_tokenizer = create_tokenizer(eng_train_text)
8
9 spanish_vocab = len(spa_text_tokenizer.word_index) + 1
10 english_vocab = len(eng_text_tokenizer.word_index) + 1
```

The extra +1 accounts for the padding index 0.

Tokenization: from words to integers

Tokenization converts a sentence into a sequence of integer identifiers.

For example:

es una broma $\longrightarrow$ [12, 45, 381]

The exact numbers depend on the tokenizer's learned vocabulary.

Neural networks do not process words directly; they process numerical tensors.

Padding

Sentences have different lengths, but neural networks process batches with fixed shapes.

Padding adds zeros after shorter sequences:

$$[12, 45, 381] \longrightarrow [12, 45, 381, 0, 0, 0, 0, 0, 0].$$

The value 0 is reserved for padding.

This is why padding-aware evaluation is necessary: padding is not a real word.

Correct tensor shapes

The notebook keeps Spanish inputs two-dimensional:

```
1 X_train.shape == (n_train, max_spanish_len)
```

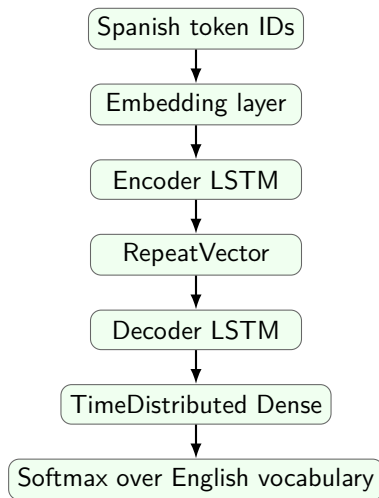
This is the correct input shape for a Keras Embedding layer.

The English targets are reshaped as:

```
1 y_train.shape == (n_train, max_english_len, 1)
```

This is convenient for `sparse_categorical_crossentropy`, where each target position contains one integer class label.

Model architecture



Model construction

The model is intentionally simple.

```
1 input_sequence = Input(shape=(max_spanish_len,), name="spanish_input")
2
3 embedding = Embedding(
4     input_dim=spanish_vocab,
5     output_dim=EMBEDDING_DIM,
6     mask_zero=True,
7     name="spanish_embedding",
8 )(input_sequence)
9
10 encoder = LSTM(LATENT_DIM, return_sequences=False)(embedding)
11 r_vec = RepeatVector(max_english_len)(encoder)
12 decoder = LSTM(LATENT_DIM, return_sequences=True,
13     dropout=DROPOUT_RATE)(r_vec)
14 logits = TimeDistributed(Dense(english_vocab))(decoder)
15 output = Activation("softmax")(logits)
```

Embedding layer

The embedding layer maps each Spanish token ID into a dense vector.

If the Spanish vocabulary size is V_s and the embedding dimension is d , then the embedding matrix has shape:

$$V_s \times d.$$

In the notebook:

- V_s is the Spanish vocabulary size;
- $d = 128$ is the embedding dimension.

The parameter `mask_zero=True` tells the LSTM to ignore Spanish padding tokens.

Encoder LSTM

The encoder LSTM reads the whole Spanish sentence and compresses it into a single vector.

Mathematically, we can represent this as:

$$h = f_{\text{enc}}(x_1, x_2, \dots, x_{T_s}),$$

where:

- $x_1, \dots, x_{T_s}$ are Spanish token embeddings;
- f_{enc} is the encoder LSTM transformation;
- h is the final encoder vector.

In this notebook, h has dimension 64.

The context-vector bottleneck

The encoder-decoder architecture compresses the full source sentence into one vector.

This is useful pedagogically, because the architecture is easy to understand.

However, it also creates a bottleneck: all information needed for translation must pass through a single latent vector.

This is one reason why attention mechanisms became important in neural machine translation.

RepeatVector

The decoder must generate an English sequence of length T_e .

RepeatVector copies the encoder vector h once for each target position:

$$h \longrightarrow (h, h, \dots, h).$$

This gives the decoder a sequence-shaped input.

The decoder then transforms these repeated context vectors into output hidden states.

The decoder LSTM produces one hidden state for each English output position:

$$z_1, z_2, \dots, z_{T_e}.$$

Here:

- z_t is the decoder hidden state at English position t ;
- T_e is the padded English sequence length.

The notebook uses `return_sequences=True` because we need an output prediction at every target time step.

TimeDistributed Dense and Softmax

At every output position, the model predicts a probability distribution over the English vocabulary.

For each decoder state z_t :

$$\hat{p}_t = \text{softmax}(Wz_t + b),$$

where:

- W is the output weight matrix;
- b is the output bias vector;
- $\hat{p}_t$ is a probability vector over English tokens.

The predicted word is usually obtained with:

$$\arg \max_k \hat{p}_{t,k}.$$

The model is compiled with sparse categorical cross-entropy.

```
1 enc_dec_model.compile(  
2     loss=sparse_categorical_crossentropy,  
3     optimizer=Adam(learning_rate=LEARNING_RATE),  
4     metrics=["accuracy", padding_aware_accuracy],  
5 )
```

This loss is appropriate because each target word is represented as a single integer token ID, not as a one-hot vector.

At each output position t , the model assigns a probability to every English token.

If the correct target token at position t is y_t , and the predicted probability of that token is $\hat{p}_{t,y_t}$, then the cross-entropy contribution is:

$$-\log \hat{p}_{t,y_t}.$$

The training loss averages this quantity over examples and target positions.

Why standard accuracy can be misleading

Standard token accuracy treats padding tokens as ordinary target tokens.

Example target:

`[<start>, its, a, joke, <end>, 0, 0, 0]`

If the model learns to predict many zeros correctly, standard accuracy may become high even if real-word translation quality is mediocre.

Therefore, the notebook uses padding-aware accuracy.

Padding-aware accuracy

Padding-aware accuracy ignores target positions where the true token is 0.

Let:

- $y_{i,t}$ be the true token for example i at position t ;
- $\hat{y}_{i,t}$ be the predicted token;
- $m_{i,t} = 1$ if $y_{i,t} \neq 0$, and $m_{i,t} = 0$ otherwise.

Then:

$$\text{Accuracy}_{\text{masked}} = \frac{\sum_{i,t} m_{i,t} \mathbf{1}(y_{i,t} = \hat{y}_{i,t})}{\sum_{i,t} m_{i,t}}.$$

Padding-aware metric: implementation idea

```
1 def padding_aware_accuracy(y_true, y_pred):
2     if len(y_true.shape) == 3:
3         y_true = tf.squeeze(y_true, axis=-1)
4
5     y_pred_ids = tf.argmax(y_pred, axis=-1, output_type=tf.int64)
6     mask = tf.not_equal(y_true, 0)
7
8     matches = tf.equal(y_true, y_pred_ids)
9     matches = tf.logical_and(matches, mask)
10
11    matches = tf.cast(matches, tf.float32)
12    mask = tf.cast(mask, tf.float32)
13
14    return tf.math.divide_no_nan(
15        tf.reduce_sum(matches),
16        tf.reduce_sum(mask)
17    )
```


Early stopping

Early stopping monitors validation loss and stops training when it no longer improves.

This has two advantages:

- it reduces unnecessary training time;
- it helps limit overfitting.

The notebook uses `restore_best_weights=True`, so the model is restored to the best validation-loss epoch.

Checkpointing

Checkpointing saves the best model observed during training.

```
1 callbacks = [  
2     EarlyStopping(  
3         monitor="val_loss",  
4         patience=5,  
5         restore_best_weights=True,  
6         verbose=1,  
7     ),  
8     ModelCheckpoint(  
9         filepath=str(best_model_path),  
10        monitor="val_loss",  
11        save_best_only=True,  
12        verbose=1,  
13    ),  
14 ]
```

This is safer than using the final epoch by default.

The model is trained on the training set, while validation data is used only for monitoring.

```
1 history = enc_dec_model.fit(  
2     X_train,  
3     y_train,  
4     validation_data=(X_val, y_val),  
5     batch_size=BATCH_SIZE,  
6     epochs=EPOCHS,  
7     callbacks=callbacks,  
8 )
```

The test set is not used during this phase.

Training history

The notebook plots:

- training loss;
- validation loss;
- standard accuracy;
- padding-aware accuracy.

These plots help answer two important questions:

- ① Is the model learning on the training data?
- ② Is validation performance improving, or is the model overfitting?

Final evaluation on the test set

The test set is used only after training is complete.

This gives a more honest estimate of how the model behaves on unseen data.

The notebook reloads the best checkpoint, then evaluates:

- test loss;
- standard token accuracy;
- padding-aware token accuracy.

Test evaluation code

```
1 if best_model_path.exists():
2     enc_dec_model = load_model(
3         best_model_path,
4         custom_objects={
5             "padding_aware_accuracy": padding_aware_accuracy
6         },
7     )
8
9 test_results = enc_dec_model.evaluate(
10     X_test,
11     y_test,
12     batch_size=BATCH_SIZE,
13     return_dict=True,
14 )
```

The custom metric must be provided when reloading the model.

From probabilities to words

The model does not directly output words.

At each English position, it outputs a probability vector over the vocabulary.

The decoding function applies:

$$\hat{y}_t = \arg \max_k \hat{p}_{t,k},$$

where:

- $\hat{p}_{t,k}$ is the probability of token k at position t ;
- $\hat{y}_t$ is the predicted token ID.

Decoding utility

```
1 def logits_to_sentence(logits, tokenizer):  
2     index_to_word = build_index_to_word(tokenizer)  
3     predicted_token_ids = np.argmax(logits, axis=-1)  
4     return token_ids_to_sentence(  
5         predicted_token_ids,  
6         index_to_word  
7     )
```

The function also removes special tokens such as <start> and stops at <end>.

Numerical metrics are not enough for translation.

The notebook therefore prints several examples from the held-out test set:

- Spanish input;
- true English sentence;
- predicted English sentence.

This helps students see typical errors, not just aggregate performance numbers.

Single-sentence inference

The notebook includes a helper function for translating a new sentence.

```
1 def translate_spanish_sentence(sentence, model=enc_dec_model):  
2     cleaned = clean_sentence(sentence)  
3     encoded = encode_and_pad(  
4         [cleaned],  
5         spa_text_tokenizer,  
6         max_spanish_len  
7     )  
8     prediction = model.predict(encoded, verbose=0)[0]  
9     return logits_to_sentence(prediction, eng_text_tokenizer)
```

This function applies the same preprocessing used during training.

Saving artifacts

A trained neural network is not enough by itself.

For future inference, we also need:

- the Spanish tokenizer;
- the English tokenizer;
- maximum sequence lengths;
- special-token definitions;
- vocabulary sizes.

Without these objects, new input sentences cannot be encoded consistently with training.

Saving model, tokenizers, and config

```
1 enc_dec_model.save(final_model_path)
2
3 with spanish_tokenizer_path.open("wb") as f:
4     pickle.dump(spa_text_tokenizer, f)
5
6 with english_tokenizer_path.open("wb") as f:
7     pickle.dump(eng_text_tokenizer, f)
8
9 config = {
10     "max_spanish_len": max_spanish_len,
11     "max_english_len": max_english_len,
12     "spanish_vocab": spanish_vocab,
13     "english_vocab": english_vocab,
14 }
```

The model and preprocessing pipeline must travel together.

The final part of the notebook shows how to reload everything in a future session.

This is important because model deployment requires reproducible inference:

- 1 load the saved model;
- 2 load the tokenizers;
- 3 load the configuration;
- 4 apply the same cleaning, tokenization, and padding procedure.